

Orquestador Stateless de Negociación A2A

Especificación Arquitectónica, Gobernanza Determinista (FSM)
y Persistencia Dual-Track en C# (.NET 9)

Equipo de Arquitectura e Ingeniería
Trabajo de Fin de Máster

7 de septiembre de 2026

Resumen

El presente documento describe la arquitectura técnica formal, la especificación de protocolos y las garantías deterministas del Orquestador Stateless de Negociación A2A. El sistema opera sobre ASP.NET Core y .NET 9, mediando transacciones comerciales automatizadas entre agentes inteligentes bajo el estándar Agent-to-Agent (A2A) mediante invocación remota JSON-RPC 2.0. Se detalla la estructura desacoplada de cinco capas, la FSM determinista en código C#, el mecanismo de compuerta con reglas duras de negocio (*Deterministic Guard*), el pipeline de validación en cascada en 4 fases, y el modelo de persistencia *Dual-Track* que combina un registro inmutable en SQL relacional con almacenamiento de ultra-baja latencia en Redis.

Índice

1. Arquitectura del Sistema: Orquestador Stateless de 5 Capas	2
1.1. Visión General y Principios de Diseño	2
1.2. Estructura de las Cinco Capas	2
1.2.1. Capa 1: Clientes A2A (Entorno Externo)	2
1.2.2. Capa 2: Gateway A2A (ASP.NET Core / C#)	2
1.2.3. Capa 3: Gobernanza y Orquestación FSM (C# / .NET 9)	3
1.2.4. Capa 4: Capa Cognitiva (LLM SDK)	3
1.2.5. Capa 5: Persistencia Dual-Track State Store	3
2. Gobernanza y FSM Determinista: Estados y Reglas de Transición	4
2.1. Definición Formal de la FSM	4
2.2. Mapeo Bidireccional de Estados	4
2.3. Ciclo de Negociación y Dinámica de Transiciones	4
2.4. Deterministic Guard y Reglas de Negocio	5
2.5. Bucle de Autocorrección Cognitiva (Self-Correction Loop)	5
2.6. Evaluador Multicriterio MCDA para Negociaciones Concurrentes	7
3. Protocolos de Comunicación: Gateway A2A y JSON-RPC 2.0	8
3.1. Especificación del Estándar A2A (Agent-to-Agent)	8
3.2. Estructura de la Agent Card (/well-known/agent.json)	8
3.3. Protocolo de Handshake y Negociación de Agent Cards	8
3.4. Métodos JSON-RPC 2.0 Expuestos por el Gateway	8
3.5. Pipeline de Validación en Cascada en 4 Fases	10
3.6. Seguridad Perimetral y Control de Acceso	10

1. Arquitectura del Sistema: Orquestador Stateless de 5 Capas

1.1. Visión General y Principios de Diseño

El sistema se concibe como un orquestador *stateless*, desacoplado y de baja latencia desarrollado sobre la plataforma ASP.NET Core y C# (.NET 9). Su propósito primordial reside en arbitrar y gobernar transacciones comerciales automatizadas entre agentes inteligentes autónomos empleando el protocolo estándar **A2A (Agent-to-Agent)** mediante mensajes estructurados bajo la especificación **JSON-RPC 2.0**.

La premisa arquitectónica directriz es la **garantía de determinismo absoluto**. Mientras que los Modelos de Lenguaje Masivos (LLM) son probabilísticos por naturaleza, las condiciones contractuales, las restricciones financieras y las transiciones del protocolo de negociación deben ser estrictamente matemáticas e inmunes a alucinaciones. Por consiguiente, la máquina de estados finitos (FSM) y los módulos de evaluación de reglas se ejecutan proceduralmente en código nativo C#, relegando la función del LLM exclusivamente a la inferencia táctica y a la generación de deltas estructurados (`facts_delta`).

1.2. Estructura de las Cinco Capas

1.2.1. Capa 1: Clientes A2A (Entorno Externo)

Constituye el estrato de agentes autónomos contrapartes (compradores o suministradores) que operan en plataformas heterogéneas. Dichos agentes:

- Inspeccionan de forma autónoma las capacidades del sistema leyendo la *Agent Card* expuesta públicamente en la ruta estandarizada `/.well-known/agent.json`.
- Inician y prosiguen las negociaciones mediante métodos formales JSON-RPC 2.0 (`create_task`, `send_message`, `get_task`).
- Respetan la alternancia estricta de turnos impuesta por el orquestador.

1.2.2. Capa 2: Gateway A2A (ASP.NET Core / C#)

Actúa como frontera perimetral de seguridad y adaptador de protocolos del sistema:

- **Seguridad Perimetral:** Aplica autenticación mutua TLS (mTLS), verificación criptográfica de cabeceras, cuotas por tenant y limitación de tasa (*rate limiting*) basada en algoritmos Token Bucket.
- **Skill Dispatcher:** Enruta los mensajes entrantes hacia las habilidades internas sin comprometer identificadores internos de infraestructura.
- **Pipeline de Validación en Cascada en 4 Fases:** Todo mensaje entrante debe superar secuencialmente cuatro filtros antes de penetrar en el orquestador:
 1. *Fase 1 (Transporte):* Verificación de integridad HTTP y conformidad estructural JSON-RPC 2.0.
 2. *Fase 2 (Esquema A2A):* Validación de esquemas (`TaskRequest`, `MessagePart`, `DataPart`).
 3. *Fase 3 (Semántica Agent Card):* Comprobación de que la habilidad solicitada y sus parámetros son compatibles con la ficha del agente.
 4. *Fase 4 (Coherencia de Ciclo de Vida):* Validación del estado de la tarea receptora para impedir operaciones fuera de turno o desincronizadas.
- **Protocolo de Handshake:** El flujo completo de autenticación mutua de tenants, consulta de lista blanca y selección dinámica de la Agent Card se encuentra formalizado en la Figura 2 (Sección 3).

1.2.3. Capa 3: Gobernanza y Orquestación FSM (C# / .NET 9)

Constituye el núcleo del sistema. Está empaquetada como librería de dominio interna en el mismo Pod de Kubernetes que el Gateway, eliminando sobrecostes de transporte de red o serialización entre capas:

- **Arquitectura 100 % Stateless:** Ninguna instancia retiene estado en memoria volátil entre peticiones. En cada solicitud entrante, el estado factual consolidado y el bloqueo de turno se rehidratan atómicamente desde el almacenamiento externo.
- **FSM Unitaria Determinista:** Define el espacio formal de estados y transiciones legales.
- **Deterministic Guard:** Motor de reglas de negocio rígidas programado en C# que evalúa márgenes mínimos de rentabilidad, stock disponible en tiempo real, límites de crédito comercial y plazos logísticos sin intervención probabilística.
- **Meta-Orquestador Fan-Out / Fan-In:** Gestiona subastas y negociaciones multilaterales concurrentes (1 comprador frente a N proveedores) utilizando primitivas asíncronas `Task.WhenAll` y aislamiento de cancelación con `CancellationTokenSource`.
- **Evaluable MCDA:** Resuelve adjudicaciones mediante un análisis multicriterio de decisión ponderado basado en una función de utilidad formal.

1.2.4. Capa 4: Capa Cognitiva (LLM SDK)

Interactúa con modelos fundacionales mediante un SDK tipado:

- Se comunica mediante *Structured Outputs* (JSON Schema estricto), produciendo un objeto `facts_delta` que resume las modificaciones cuantitativas de la propuesta.
- Dispone de un **Bucle de Autocorrección** (*Self-Correction Loop*): si la propuesta es rechazada sintácticamente o por transición ilegal (HTTP 422), el agente reinyecta el error en su prompt con un límite de reintentos $N \in \{2, 3\}$. Si se agota el umbral, la sesión se congela automáticamente.

1.2.5. Capa 5: Persistencia Dual-Track State Store

El sistema desacopla intencionadamente las necesidades analíticas y legales de las necesidades de control de concurrencia en tiempo real:

- **Track 1 (Raw Transcript Store - Base de Datos SQL Relacional):** Bitácora inmutable, cronológica y *append-only* que almacena cada mensaje literal, firma criptográfica, emisor y marca de tiempo. Diseñada para auditoría legal, forense y cumplimiento contractual.
- **Track 2 (Hot State Store - Redis In-Memory):** Almacena el *Fact Sheet* consolidado (SKUs, precios acordados, condiciones Incoterms, concesiones) y gobierna los cerrojos de concurrencia y alternancia de turnos con latencias inferiores a 2 ms.

La interacción integral entre las capas del orquestador, los bloqueos de turno en Redis Hot State, la evaluación del Deterministic Guard y las transiciones deterministas de la FSM a lo largo del ciclo de vida se ilustra formalmente en la Figura 1 (Sección 2).

2. Gobernanza y FSM Determinista: Estados y Reglas de Transición

2.1. Definición Formal de la FSM

La máquina de estados finitos que gobierna la negociación se define formalmente como una tupla:

$$\mathcal{M} = (S, \Sigma, \delta, s_0, F) \quad (1)$$

donde:

- $S = \{\text{on_board}, \text{board_format_err}, \text{validation}, \text{val_format_err}, \text{ended}, \text{frozen}\}$ representa el conjunto de estados internos admisibles.
- Σ representa el alfabeto de eventos y transiciones provocadas por mensajes entrantes o resultados de evaluación de reglas.
- $\delta : S \times \Sigma \rightarrow S$ es la función de transición determinista ejecutada proceduralmente en C#.
- $s_0 = \text{on_board}$ es el estado inicial tras completarse con éxito el handshake.
- $F = \{\text{ended}, \text{frozen}\}$ es el conjunto de estados terminales.

2.2. Mapeo Bidireccional de Estados

El estándar público A2A define estados genéricos para las tareas de agentes. El orquestador mantiene un mapeo bidireccional estricto entre estos estados y los estados del motor interno de C#, tal como se sintetiza en el Cuadro 1.

Estado A2A (Público)	Estado FSM (C#)	Naturaleza	Significado y Comportamiento
working	on_board	Transitorio	Fase activa de negociación, diálogo persuasivo e intercambio de contraofertas.
input_required	board_format_err	Recuperable	Payload inválido sintácticamente; activa el bucle de autocorrección cognitiva.
evaluating	validation	Auditoría	Evaluación de la propuesta consolidada frente al Deterministic Guard.
input_required	val_format_err	Recuperable	Discrepancia de esquema de datos entre el Fact Sheet y el backend de negocio.
completed	ended	Terminal	Acuerdo formalmente alcanzado, validado por ambas partes y firmado.
failed	frozen	Terminal / Pausa	Negociación congelada tras superar el límite de reintentos ($N > 3$) o rechazo definitivo.

Tabla 1: Mapeo Bidireccional entre el estándar A2A y la FSM interna del Orquestador.

2.3. Ciclo de Negociación y Dinámica de Transiciones

La interacción dinámica entre los agentes negociadores se rige por una secuencia estricta de fases operativas y salvaguardas procedurales, tal como se ilustra en la Figura 1. El ciclo se estructura en tres bloques fundamentales:

1. **Bloque 1: Negociación Activa (on_board, board_format_err, frozen):** Constituye el diálogo comercial recurrente. El agente proponente transmite una oferta inicial en estado **on_board**. La contraparte puede responder aceptando o emitiendo una contraoferta en el

mismo estado. Si se detecta un error de sintaxis o de esquema en el payload, el orquestador retorna **board_format_err**, obligando al agente a subsanar el payload e intentar de nuevo. Si concurren condiciones de espera externa o bloqueo transitorio, la negociación se suspende temporalmente en **frozen** hasta su reanudación controlada.

2. **Bloque 2: Validación de Negocio (validation, val_format_err):** Al converger hacia un acuerdo preliminar satisfactorio, se emite una solicitud en estado **validation** para someter el *Fact Sheet* al *Deterministic Guard*. Si la estructura del mensaje de validación es errónea, se activa el estado **val_format_err** para corrección. Si las reglas duras de negocio (márgenes comerciales, disponibilidad de inventario o límites de crédito) son infringidas, el orquestador rechaza el cierre y retrotrae la negociación al estado **on_board** para su reformulación.
3. **Bloque 3: Finalización Contractual (ended):** Una vez que ambas partes han validado satisfactoriamente los términos cuantitativos y legales frente a sus respectivos backends, la FSM transiciona de forma irreversible al estado terminal **ended**, procediendo a la firma del acuerdo y al archivado de la bitácora en la persistencia *Dual-Track*.

2.4. Deterministic Guard y Reglas de Negocio

El componente *Deterministic Guard* actúa como una compuerta infranqueable ante cualquier acuerdo propuesto por los agentes. Su evaluación es binaria y determinista, evaluando las siguientes restricciones operativas:

1. **Margen Mínimo Comercial:** Comprueba que el precio ofertado P_{acuerdo} respete el umbral mínimo de rentabilidad según el costo base C_{base} :

$$\frac{P_{\text{acuerdo}} - C_{\text{base}}}{P_{\text{acuerdo}}} \geq \mu_{\text{mín}} \quad (2)$$

2. **Disponibilidad de Inventario:** Verifica contra el sistema de gestión (ERP) que la cantidad demandada Q no exceda el stock físico comprometible S_{disp} .
3. **Límite de Riesgo de Crédito:** Evalúa la exposición crediticia del tenant solicitante frente a su póliza vigente.
4. **Factibilidad de Plazo:** Comprueba que la fecha límite de entrega T_{entrega} sea compatible con el tiempo de transporte mínimo T_{lead} .

Si cualquiera de las condiciones anteriores resulta insatisfecha, la transición hacia **ended** es abortada y la FSM retorna a **on_board** adjuntando un diagnóstico cuantitativo para renegociación.

2.5. Bucle de Autocorrección Cognitiva (Self-Correction Loop)

Cuando un mensaje recibido adolece de errores de sintaxis, no respeta el esquema JSON esperado o intenta un salto de estado ilegal según la matriz δ , el Gateway responde con un código de error HTTP 422 (*Unprocessable Entity*).

El agente cognoscitivo receptor debe implementar el bucle formal de autocorrección:

1. Captura del objeto de diagnóstico retornado por el orquestador.
2. Inyección del mensaje de error en el contexto conversacional del LLM.
3. Regeneración estricta de la propuesta mediante Structured Outputs.
4. Limitación imperativa: el contador de reintentos acumulados n está acotado por $n \leq N_{\text{máx}}$ (con $N_{\text{máx}} = 3$). En caso de alcanzarse $n > N_{\text{máx}}$, el orquestador transiciona forzosamente la sesión al estado **frozen (failed)** y deriva la incidencia para mediación humana.

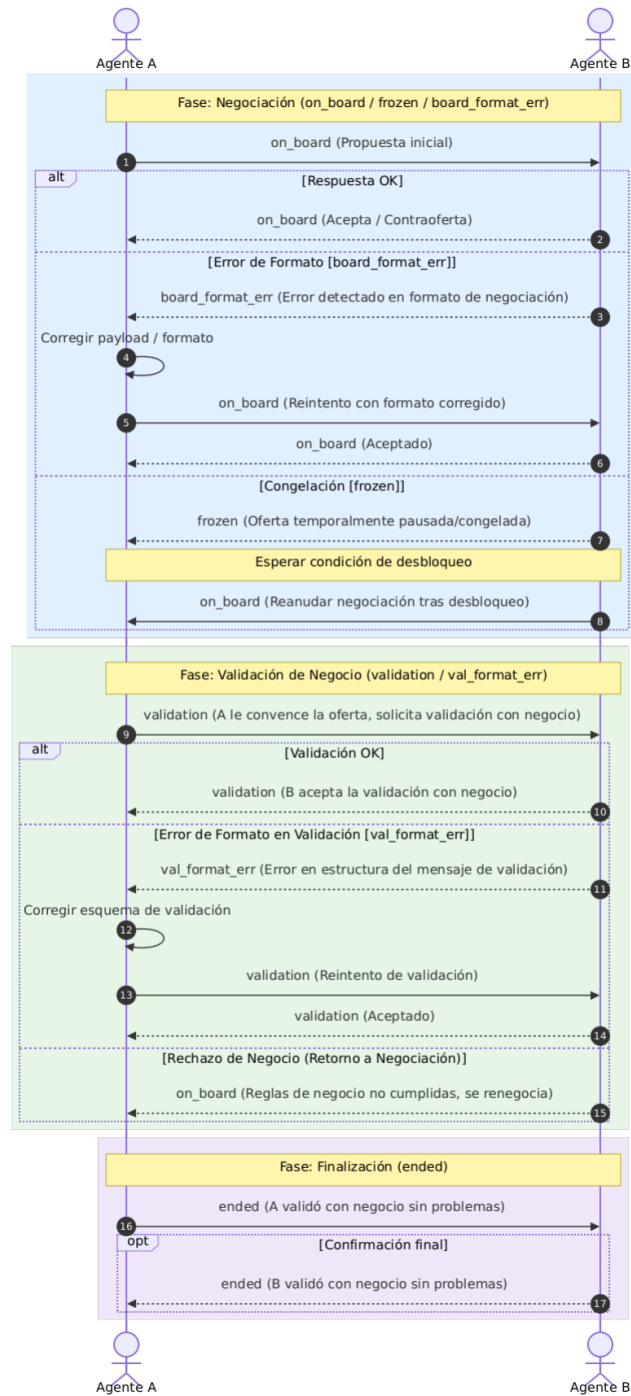


Figura 1: Ciclo de negociación gobernado por la FSM determinista: alternancia de turnos y diálogo en `on_board`, bucle de autocorrección sintáctica (`board_format_err`), suspensión temporal (`frozen`), compuerta de validación contra reglas de negocio (`validation`, `val_format_err`) y formalización final (`ended`).

2.6. Evaluador Multicriterio MCDA para Negociaciones Concurrentes

En escenarios multilaterales (subastas con N proveedores concurrentes), el Meta-Orquestador resuelve la selección del proveedor adjudicatario empleando un modelo MCDA (*Multi-Criteria Decision Analysis*) basado en una función de utilidad aditiva ponderada:

$$\text{Score}_k = w_p \cdot \tilde{P}_k + w_t \cdot \tilde{T}_k + w_d \cdot \tilde{D}_k + w_r \cdot \tilde{R}_k \quad (3)$$

donde $\sum w_i = 1$, y los términos representan valores normalizados en el intervalo $[0, 1]$ correspondientes a Precio (P), Tiempo de entrega (T), Descuentos por volumen (D) y Reputación histórica del proveedor (R). El ganador recibe la notificación `complete_task`, mientras que los proveedores no seleccionados reciben `cancel_task`.

3. Protocolos de Comunicación: Gateway A2A y JSON-RPC 2.0

3.1. Especificación del Estándar A2A (Agent-to-Agent)

El estándar A2A establece un marco formal e interoperable para la comunicación entre sistemas multiagente autónomos. Sobre la capa de transporte seguro (HTTPS / mTLS), el intercambio de mensajes se articula mediante el protocolo de invocación remota JSON-RPC 2.0.

3.2. Estructura de la Agent Card (/well-known/agent.json)

El punto de anclaje semántico del sistema es su Agent Card, expuesta públicamente en la ruta estandarizada `/well-known/agent.json`. Esta especificación describe las habilidades (*skills*) ofrecidas por el orquestador, los esquemas de entrada y salida, así como los requerimientos criptográficos de autenticación para cada operación soportada.

3.3. Protocolo de Handshake y Negociación de Agent Cards

Antes de dar inicio formal a una negociación comercial, los agentes y sus respectivas entidades (*tenants*) deben establecer un canal seguro de confianza mutua mediante el protocolo de *handshake* (apretón de manos). Este procedimiento garantiza que únicamente contrapartes autorizadas y verificadas puedan acceder al catálogo de servicios y comenzar el intercambio de propuestas.

Tal como se detalla en la Figura 2, el flujo opera a través de las siguientes fases secuenciales:

1. **Solicitud de Conexión y Validación de Identidad:** El *Tenant A* manifiesta su intención de negociar con el *Tenant B* a través del software orquestador. El sistema procede inmediatamente a validar las credenciales de identidad del *Agente A*. En caso de discrepancia criptográfica o firma inválida, la sesión se aborta con error terminal.
2. **Verificación en Lista Blanca (Whitelist):** El orquestador consulta al *Tenant B* para verificar si el *Tenant A* figura en su lista de confianza permitida. Si la verificación resulta desfavorable, la comunicación se clausura impidiendo accesos no autorizados.
3. **Entrega del Catálogo de Agent Cards:** Una vez aprobada la conexión, el *Tenant B* suministra su catálogo de tarjetas de agente (*Agent Cards*) disponibles. El orquestador transmite dicho catálogo al *Agente A*.
4. **Selección de Habilidad y Validación Cruzada:** El *Agente A* inspecciona el catálogo y selecciona la tarjeta de agente idónea para la transacción. Si el agente requerido no está disponible, el flujo finaliza con estado **DECLINE**. Si existe, se emite una orden de selección (**SELECT**), tras la cual ambas partes ejecutan una verificación mutua de credenciales con el *Agente B* antes de dar por completado el apretón de manos y transicionar al estado operativo inicial **on_board**.

3.4. Métodos JSON-RPC 2.0 Expuestos por el Gateway

El Gateway implementa cuatro primitivas principales de negociación:

- **create_task:** Inicializa una sesión de negociación multilateral o bilateral. Recibe el identificador del tenant, la habilidad solicitada y el payload inicial.
- **send_message:** Transmite un turno de negociación conteniendo el **facts_delta** cuantitativo y el mensaje contextual hacia la contraparte.
- **get_task:** Consulta el estado actual de la tarea, el emisor del turno activo y el resumen factual consolidado en Redis.

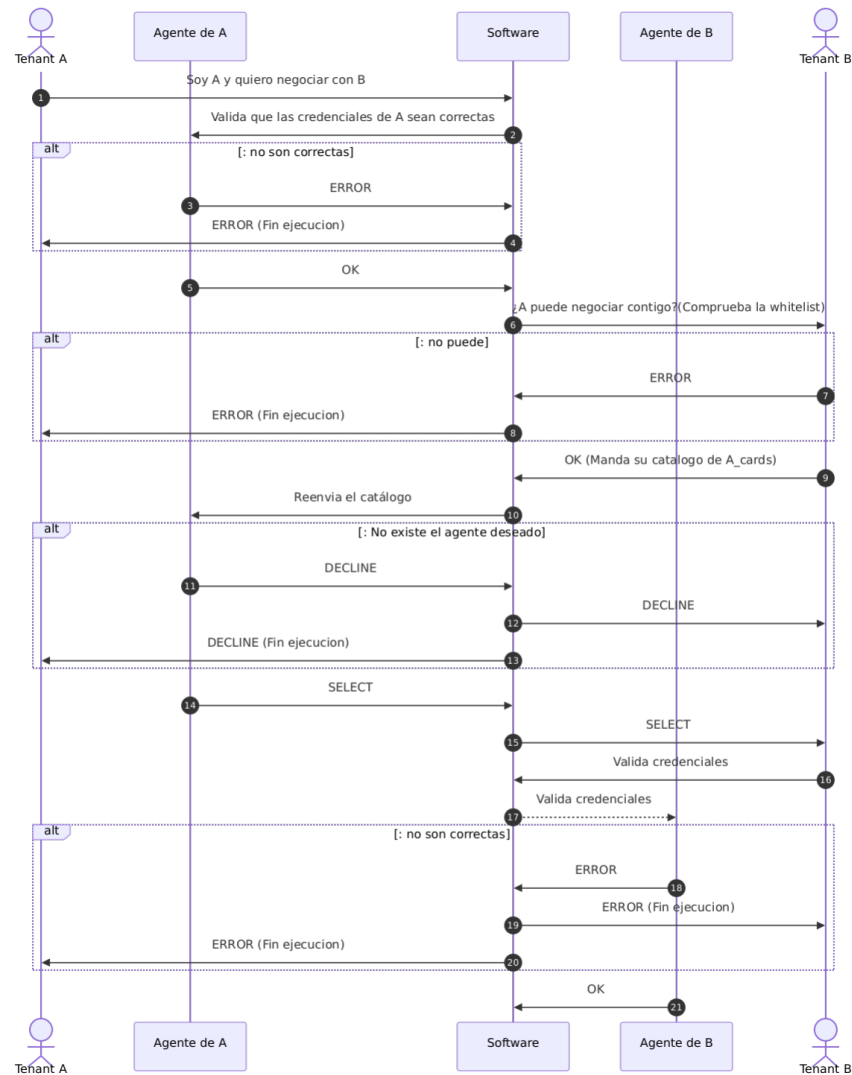


Figura 2: Secuencia de Handshake bilateral: autenticación de tenants, validación en lista blanca (*whitelist*), distribución del catálogo de Agent Cards y verificación cruzada de credenciales criptográficas.

- **cancel_task:** Provoca la cancelación inmediata de una negociación activa, transicionando a `frozen` (`failed`).

3.5. Pipeline de Validación en Cascada en 4 Fases

A fin de asegurar la robustez del sistema frente a mensajes malformados o ataques perimetrales, el Gateway procesa cada petición a través de cuatro filtros secuenciales:

1. **Fase 1: Validación de Transporte y Protocolo Base:** Comprueba método HTTP POST, cabeceras requeridas (`application/json`) y conformidad estructural JSON-RPC 2.0.
2. **Fase 2: Conformidad de Esquema A2A:** Valida que los parámetros correspondan a un `TaskRequest` o `MessagePart` conforme al metamodelo A2A.
3. **Fase 3: Validación Semántica de Agent Card:** Verifica que la habilidad solicitada exista en la Agent Card activa y satisfaga su esquema de entrada.
4. **Fase 4: Coherencia de Ciclo de Vida y Turnos:** Consulta atómicamente en Redis el estado de la tarea y la titularidad del turno activo para evitar operaciones desincronizadas.

3.6. Seguridad Perimetral y Control de Acceso

El Gateway aplica directivas perimetrales antes de procesar el payload:

- **Autenticación mTLS:** Validación mutua de certificados X.509 para comprobar la identidad del tenant emisor.
- **Rate Limiting y Cuotas:** Implementación de limitación de tasa por IP y por Tenant ID mediante algoritmo Token Bucket en Redis, previniendo denegación de servicio.